

ABSuite — Complete Dossier

****What it is, what it does, who pays for it, what it is worth, and what is not yet true.****

Prepared 28 August 2026 · Enock Labs (Themba Mpehle)

How to read this document

Every claim here is one of three kinds, and they are never mixed:

Mark	Meaning
MEASURED	Taken from this repository by running something. Reproducible with the command given.
CITED	From a named outside source, with the source. Their number, not ours.
REASONED	An inference or arithmetic scenario. The method is shown so it can be argued with.

Nothing in this document is a forecast presented as a fact. Where a range is given, the assumptions are stated beside it so a reader can substitute their own.

1. Executive summary

ABSuite is infrastructure for a problem that arrived faster than the tools for it: **organisations are giving AI agents the authority to act, and cannot prove afterwards what those agents did.**

- **CITED** — 80% of organisations deploying AI agents have no mature governance model for them. (prefactor.tech, 2026)
- **CITED** — The EU AI Act requires high-risk AI systems to keep **tamper-evident logs for a minimum of six months**, with full traceability of every algorithmically driven decision. ([Article 12](#))
- **MEASURED** — ABSuite produces exactly that: signed, hash-chained execution records, verifiable by a third party holding only a public key.

The product is **built and tested to an unusual standard** (1,063 tests, 28 enforced build gates, MIT licensed, published on npm). It has **no paying customers**, and the single thing standing between it and its first invoice is a deployment, not a feature.

The commercial insight that matters: because the code is MIT, no feature can be a moat — anyone may run all of it free. What cannot be copied is **being a disinterested third party**. The paid tiers sell an outside witness to your records. A fork cannot replicate that, because a fork's witness is equally self-interested toward its own users.

2. What ABSuite is, in one paragraph

A trust layer that sits between an AI agent and the systems it acts on. It replaces the long-lived API key most agents are given with a **narrow, expiring, revocable capability token, and writes a signed, hash-chained record** of everything that token was used for. Anyone can verify those records — including someone with no reason to trust the operator — because verification needs only a public key, and a public key cannot forge.

3. The problem

3.1 What people actually do today

An engineer building an agent gives it an API key. That key usually has full account access, because scoping it is work and the agent "needs to be able to do things".

The consequences are structural, not hypothetical:

- **The blast radius is the whole account.** A prompt injection, a hallucinated tool call or an ordinary bug can do anything the key can do.
- **You find out from the damage.** There is no record of intent, authority or sequence — only application logs, which the same party can edit.
- **You cannot prove innocence either.** When a customer disputes what happened, an ordinary log is not evidence. It is an assertion by the accused.

3.2 Why ordinary logging does not close it

A log file proves nothing to a hostile reader. The operator holds the write access. A sufficiently motivated party could produce a perfectly plausible log today and claim it was written last year.

Cryptographic chaining fixes half of that: it proves nobody edited a record **after** it was written. It cannot prove **when** it was written, because the operator also holds the signing key.

****Nothing inside a single deployment can close that gap, because everything inside it is signed by the same party.**** That is not an implementation weakness — it is a property of the situation, and it is the reason the paid product exists.

4. What ABSuite does — measured

Every row below is code in this repository. Reproduce with

```
pnpm build && pnpm test.
```

Capability	What it actually does
Capability tokens	HS256 JWTs, <code>node:crypto</code> only — no third-party JWT dependency on the security path. Scopes match segment-wise: <code>read:*</code> grants <code>read:users</code> , never <code>read:users:delete</code> . <code>alg: none</code> downgrades and tampered payloads are rejected.
Expiry and audience	Tokens carry <code>expiresIn</code> and <code>audience</code> ; validation returns a specific reason (<code>TOKEN_EXPIRED</code> , <code>CAPABILITY_INSUFFICIENT</code> , ...) rather than a boolean.
Revocation	Shared store across services. If the revocation store is unreachable it returns 503 rather than failing open.
Signed execution records	Ed25519. An auditor can verify holding only a public key — and therefore cannot also forge.
Hash chaining	Every record links to its predecessor. <code>verifyChain()</code> names the first broken record by id and reason, and distinguishes an edited record from a rotated key.
Chain checkpoints	Signed notes shortening re-verification, reported as a weaker claim than a full walk — deliberately, and the result says which it is.
Retention with signed anchors	Records past a window are removed and replaced by a signed anchor , so the surviving chain still verifies. Without it, retention and truncation are indistinguishable.
Audit export	A file an auditor verifies without ABSuite — no database, no network, no server. Carries records, links, signatures, the retention anchor and any notary receipts.
Third-party notarisation	An outside notary witnesses the chain head on a schedule and returns a signed receipt. This is the part a deployment cannot do for itself.
Chain-break alerting	Fires on the transition , not the state, so a standing problem does not send an alert every sweep until somebody mutes it.
Trust scoring	Pure functions over recorded events. 30-day half-life. No score about a human without explicit opt-in.
Governance	The rule that permitted an action is recorded alongside the action.
Scheduling / task queue	<code>edge-run</code> — cron, retries, self-healing, every mutating endpoint capability-guarded.
MCP server	Capability-checked, attested tool calls over Model Context Protocol.
Benchmarking	<code>quickbench</code> — percentile latency and statistically grounded regression detection.

4.1 Scale and quality — MEASURED

Source lines (excluding tests)	22,349
Test lines	11,787
Tests	1,063 across 49 suites
Gates run by <code>pnpm verify</code>	27
<code>check:*</code> scripts in total	28 (one, <code>check:live</code> , runs against a deployed instance)
HTTP routes, documented and drift-checked	131

npm packages published	8
Licence	MIT

The gates are the unusual part. They are not linting. Examples that ran today and caught real defects:

- `check:no-fabrication` — scans interface source for invented data
 - `check:numbers` — every published figure must match the repository
 - `check:routeauth` — every route guarded or explicitly declared public ****with a reason****
 - `check:metered` — the billing screen's claim about which quotas are counted must match the code that counts them
 - `check:outbound` — every outbound fetch guarded or annotated with why it is safe
-

5. How it works

5.1 The eight layers — MEASURED

pnpm gen: layers reads the repository and reports:

#	Layer	Status
1	Identity	BUILT
2	Capability	BUILT
3	Evidence	BUILT
4	Trust	BUILT
5	Governance	BUILT
6	Autonomy	BUILT
7	Collective Intelligence	not built
8	Civilization	not built

Six of eight. The two unbuilt layers are named as unbuilt in the code and in the generated architecture file — they are a roadmap, not a claim.

5.2 The packages

Package	What it is for
@absuitecore/capkit	The core. Tokens, records, chain, retention, export.
@absuitecore/trust	Evidence-based trust scoring for multi-agent systems.
@absuitecore/notary	A disinterested witness to a chain head. The commercial keystone.
@absuitecore/edge-run	Cron, queueing, retries — every mutating endpoint capability-guarded.
@absuitecore/mcp	Capability-checked tool calls over MCP.
@absuitecore/quickbench	Latency percentiles and regression detection.
@absuitecore/connector-starter	Connector registry, credential verification, scaffolding.
@absuitecore/cli	Unified command line.
dashboard-ui	The Trust Operations Center.

5.3 The notary, and why it is separate

A notary receives **32 bytes** — a hash — and returns a signed receipt saying

I saw this value at this time. It never sees a record. It cannot verify a chain and does not try.

One receipt is a timestamp. **A series is an external, ordered witness.** A chain is append-only, so every head a notary ever saw must still be in it, at the same position, forever. An operator who rewrites history produces a chain that verifies perfectly against itself and fails against evidence held by somebody

with no stake in the answer.

Witnessing is deliberately **unauthenticated**. Anyone may submit any hash. A notary able to refuse to witness somebody would have exactly the power a disinterested party must not have.

6. What is built, and what is not — the honest column

This section exists because the same product had two advertised features that did not exist as recently as this morning. Both were found and removed.

6.1 Built and running

Everything in section 4.

6.2 Built, never tested against the real counterparty

	State
PayPal signature verification	Correct against generated keys and known vectors. Has never verified a signature PayPal actually produced.
PayPal webhook route	Refuses correctly over real HTTP. Has never accepted a real event.
Notary witnessing client	Refuses correctly. Has never talked to a live notary.

Until a deployed instance takes a real event, *"it works"* is a reading of a specification, not a measurement. This is written into the test files themselves.

6.3 Not built

- **SAML SSO** — removed from the price list today. It was advertised and did not exist anywhere except the features array.
- **Layers 7 and 8** — named as unbuilt.
- **A hosted, paying instance** — nothing is deployed.

6.4 Known limitations, stated

- **One hash chain per instance**, shared by all tenants. Per-tenant retention and per-tenant witnessing cadence are therefore impossible without a schema change. Today the instance takes the *longest* retention and the *shortest* witnessing interval any tenant is owed — erring toward over-serving in both.
 - **Team's 90-day retention is below the EU AI Act's six-month minimum.** See §8.3. This is a live product decision, not a defect.
 - **Trust scoring of humans requires explicit opt-in** and is a constitutional refusal by default.
-

7. The moat

7.1 MIT means a feature can never be the moat

The licence permits anyone to take the whole codebase, delete the quota enforcement, run every tier and sell the result in competition. That is deliberate. ****Any rung of a price ladder defined by withheld code is a rung somebody rebuilds in an afternoon.****

This is not a weakness to be fixed by relicensing. The permissive licence is the distribution: it is why the packages were installed at all, and restricting the terms now would trade the only asset the project has.

7.2 What cannot be copied is not code

The notary's value is that it is somebody else.

A customer may run their own — the package is MIT and in the repository. What they get is their own signature vouching for their own chain, which proves nothing to the auditor the exercise exists for.

A competitor forking ABSuite cannot solve this either: their notary is equally self-interested toward their own users. ****Disinterest is a position in the world, not a feature in a repository****, and it is the one thing on the price list that cannot be implemented away.

It also has a network property: ****a widely-held witness is better evidence than an obscure one.**** Every additional customer makes the receipts more valuable to every other customer — the only compounding asset in the product.

7.3 The trademark position

MIT licenses the code and is silent on the name. That silence is now closed:

"ABSuite", "CapKit" and "Enock Labs" are asserted as marks. Anyone may fork, improve and sell — **under their own name**. This is the same posture Redis and Elastic held for years while their code stayed permissive.

8. Where the world is — the market context

8.1 Agents are being deployed faster than they are being governed — CITED

Enterprises running AI agents in production	57%+ (sqmagazine)
Enterprise apps shipped in Q1 2026 embedding at least one agent	80% (digitalapplied)
Gartner: enterprise applications with task-specific agents by 2026	40%, up from <5% in 2025 (accelerate)
Organisations using AI in at least one function (McKinsey)	88%
...of those, scaling an agentic system	23%
Organisations with a mature governance model for autonomous agents	1 in 5 (prefactor)

That last row is the market, stated by someone else. Four in five organisations deploying agents are doing so without the governance infrastructure to manage them. The gap between deployment and control is the space this product occupies.

8.2 The governance market is small, real, and growing very fast — CITED

Estimates vary by scope. All of them agree on the direction.

Source	2026	Later	CAGR
AI TRISM (MarketsandMarkets)	\$3.09B	\$11.61B by 2031	30.3%
AI Governance (GM Insights)	\$1.1B	\$13.1B by 2035	31.4%
AI Governance (Grand View)	—	\$3.59B by 2033	36.0%
AI Governance (Mordor)	\$0.44B	\$1.51B by 2031	28.2%
Enterprise AI Governance & Compliance (FMI)	\$2.55B	\$11.05B by 2036	15.8%

REASONED — what this actually means for a one-person company: the spread between \$0.44B and \$3.09B for the same year shows these are scope definitions rather than measurements, and none of them should be used to justify anything. The useful reading is directional: a market growing at 28–36% compounding is one where buyers have budget lines that did not exist two years ago. **A market being large is not a reason anyone buys from you.** It is only a reason the buyer exists at all.

8.3 Regulation is the forcing function — CITED, and this is the strongest signal

The EU AI Act, [Article 12](#), requires every high-risk AI system to be designed with **automatic event logging** allowing ****full traceability of the system's operation from deployment through decommissioning, covering every algorithmically driven decision.****

The core obligation is described by practitioners as:

tamper-evident logging retained for six months minimum

— [Help Net Security](#), [netguardia](#)

****That sentence is a product specification for ABSuite, written by a legislature.****

Timing, following the AI Act Omnibus provisional agreement of 7 May 2026:

Category	Deadline
Annex III systems (recruitment, credit scoring, law enforcement)	2 December 2027
Annex I systems embedded in regulated products	2 August 2028

REASONED — two consequences, and one is actionable today:

— **The deferral is good news, not bad.** It moves the buying window from "already late" to "eighteen months of budgeted preparation", which is precisely when infrastructure gets bought rather than improvised.

— **Team's 90-day retention is below the six-month floor.** A customer cannot use the \$49 tier to satisfy Article 12. Either Team moves to 180+ days, or Business is positioned explicitly as *the compliance tier* — which is a stronger sales position and needs no code change. ****This is a decision to make deliberately.****

9. Who the buyers are

Ordered by how sharply they feel the pain today. This is **REASONED** from the capability set and the regulatory position, not from customer interviews — there have been none, and that is stated plainly in §12.

9.1 Primary — agents already touching money, records or infrastructure

Who: fintech, insurtech, healthtech, logistics, any company where an agent approves a payment, changes a record, or provisions something.

The trigger: a security review has stopped, or will stop, an agent rollout.

Someone senior asked *"what can it do if it goes wrong, and how would we know?"* and the honest answer was *"anything, and we wouldn't."*

Why they pay: they cannot ship without an answer. The alternative is building this themselves — three to six engineer-months for something they do not want to own.

Where they buy: Business tier. The audit export is the artefact their compliance function actually asked for.

9.2 Primary — anyone inside EU AI Act scope

Who: providers and deployers of Annex III systems — recruitment, credit scoring, education, essential services, law enforcement.

The trigger: December 2027, and the internal programme that starts eighteen months before it.

Why they pay: Article 12 is not optional and tamper-evident is not a log file. They will buy something. The only question is whether it is a \$2,990/year line item or a consultancy engagement costing fifty times that.

Where they buy: Business, annual. ****This is the highest-value segment and it has a date on it.****

9.3 Secondary — platforms running agents on behalf of other people's customers

Who: agent platforms, AI automation vendors, vertical SaaS embedding agents.

The trigger: their own customers ask *"how do I know what your agent did in my account?"* — a question they currently cannot answer.

Why they pay: it becomes a feature they resell. Per-tenant isolation they can demonstrate rather than assert.

Where they buy: Business, and eventually Enterprise. Multi-tenant chains are the schema change §6.4 names.

9.4 Secondary — regulated industries piloting agents

CITED: banking and insurance lead agent production deployment at **47%**; healthcare **18%**, government **14%** ([sqmagazine](#)).

REASONED: the leaders buy soonest; the laggards buy largest, because their procurement is slower and their obligations heavier. Healthcare and government at 14–18% are not a weak market — they are an early one.

9.5 The developer who becomes the buyer

Who: the individual engineer who installs capkit from npm because they did not want to hand an agent a root key.

Why they matter more than their spend: they cost nothing to serve (MIT, self-hosted, unmetered) and they are the person in the room when their employer asks the security question. ****Bottom-up adoption is the distribution channel, and the free tier is the marketing budget.****

9.6 Who is NOT the buyer

Stated because chasing them wastes the scarcest resource here, which is time:

- **Hobbyists and side projects.** They self-host, correctly and permanently.
- **Companies with no agent in production.** No pain, no purchase.
- **Anyone wanting a full AI observability suite.** ABSuite proves what happened;

it does not trace tokens, tune prompts or evaluate model quality. Competing there means competing with funded platforms on their ground.

10. Pricing and the revenue model

10.1 The ladder — MEASURED, from `billing.ts`

	Free	Team	Business	Enterprise
Monthly	\$0	\$49	\$299	negotiated
Annual	—	\$490	\$2,990	negotiated
Annual saving	—	\$98 (2 months)	\$598 (2 months)	—
Witnessed by us	never	daily	hourly	hourly
Rewrite window	<i>unwitnessed</i>	24 hours	1 hour	1 hour
Agents	3	25	250	unlimited
Validations / month	10,000	500,000	5,000,000	unlimited
Audit retention	forever, self-hosted	90 days	365 days	7 years
Cross-service revocation	—	✓	✓	✓
Chain-break alerting	—	✓	✓	✓
Verifiable audit export	—	—	✓	✓

The rewrite window is the product. A chain witnessed hourly can be rewritten within an hour and no further. That is a number a compliance officer can put in a document, and it is the one line on this table a competitor cannot implement.

10.2 What is actually being sold

Not features — MIT gives those away. **Operation:** uptime, upgrades, the alert genuinely arriving at three in the morning, a person to call, and above all a witness with no stake in the answer.

10.3 Why annual matters more than the discount costs

REASONED. Two months free is a ~17% discount. For a company with no runway, **one Business annual is \$2,990 received in one payment.** The discount is not a concession, it is the price of converting revenue into runway.

11. Revenue arithmetic

This is not a forecast. It is arithmetic on assumptions that are written down so they can be replaced. Nobody has bought anything; any projection would be invention, and invention next to a price is the defect this whole codebase has gates against.

11.1 What one customer is worth

	Monthly	Annual	3-year value at annual
Team	\$49	\$490	\$1,470
Business	\$299	\$2,990	\$8,970

11.2 Scenarios — REASONED, with the assumption stated

Read these as "*what would have to be true*", not as "*what will happen*".

Scenario	Assumption	Annual recurring revenue
First blood	1 Business, 3 Team	\$4,460
Ramen	5 Business, 10 Team	\$19,850
One person, full time	15 Business, 25 Team	\$57,100
Small company	50 Business, 100 Team	\$198,500
Category foothold	200 Business, 400 Team	\$794,000

REASONED — what these actually say. *Ramen* — the point at which this replaces a salary in South African terms — is **fifteen customers**. Not fifteen thousand. Fifteen. That is the honest scale of the thing, and it is why the absence of a sales motion is the binding constraint rather than the size of the market.

And the reverse reading matters more: at \$2,990 a year, a *single* EU AI Act customer covers hosting, the domain, and a year of infrastructure, several times over. The first customer is not a rounding error; it changes the operating position entirely.

11.3 Cost to serve — MEASURED and REASONED

Hosting one instance (Fly, always-on, 1GB)	REASONED ~\$5–7/month; scale-to-zero reduces it to cents while idle
Notary (32 bytes in, a signature out)	REASONED negligible; it is the cheapest service in the product
Marginal cost of one more customer	REASONED near zero until multi-tenant chains are needed
Payment processing	PayPal fees, ~3.5%+ cross-border

Gross margin is not the problem here. Distribution is.

11.4 The honest read on current traction — MEASURED

npm downloads, last 30 days:

Package	Downloads
capkit	2,247
quickbench	1,159
edge-run	1,065
connector-starter	969
cli	957
trust	926
mcp	841
notary	289

And the necessary correction. August for capkit: 1,614 downloads, of which **1,323 fell on two days**. The other 26 days total 291, a ****median of 8 per day**. **The last seven days:** 21 in total.**

REASONED: the spike days are mirrors, scrapers and CI following a publish.

The organic signal is single digits per day. ****These are not thousands of adopters, and treating them as demand would be the most expensive mistake available.**** They are evidence that the packages exist and resolve — nothing more. Real traction has not started because nobody has been asked to buy anything.

12. Risks, stated plainly

Risk	Severity	Honest assessment
No customer has ever been asked	Highest	There is no sales motion, no landing page, no outbound. Everything else on this list is secondary to it.
Never tested against a real counterparty	High	PayPal and the notary have never handled a live event. Both are refusals-only tested. One deployment resolves it.
Single-person dependency	High	One author. No bus factor. Mitigated only by the unusually high test and gate coverage.
MIT permits direct competition	Medium	Mitigated structurally by the notary and legally by the trademark, not by the licence.
Team retention below the EU AI Act floor	Medium	§8.3. Fixable in one number or one positioning sentence.
Single shared hash chain	Medium	Blocks true per-tenant retention and cadence. A schema change, scoped and understood.
Payment concentration on PayPal	Medium	One provider can freeze everything on an algorithm. The account was limited within an hour of creation on 28 Aug and is under review.
Stripe unavailable in South Africa	Low, but structural	Confirmed. Constrains provider choice permanently unless incorporation moves.
Larger vendors entering	Low today	Observability platforms trace agents; they do not produce independently verifiable evidence. Different claim, different buyer.
The market may not buy from a one-person company	Real	Compliance buyers ask about vendor viability. The audit export is the mitigation: their evidence survives even if the vendor does not, because verification needs only a public key.

That last mitigation is worth stating as a selling point rather than a defence. A customer's records remain verifiable if Enock Labs disappears entirely. Very few vendors in this category can say that, and it is a direct consequence of the MIT licence and the export format.

13. What has to happen next

In order. Nothing below is a feature.

#	Action	Blocked on	Cost
1	Deploy an instance and a notary	A public URL — Render free from 1 Sept, or a card on Fly	~\$0–7/month
2	Fire a real PayPal event at it	(1)	Nothing
3	Witness a real chain head	(1), and the notary being a <i>separate</i> deployment	Nothing
4	Decide the Team retention question	A decision, not code	Nothing
5	A landing page saying the sentence in §14	Nothing	Nothing
6	Ask ten people to buy it	Nothing at all	Nothing

Step 6 has been available since before this document was written. Every other line exists to make step 6 easier, and none of them substitutes for it.

14. The sentence

Anyone can run ABSuite free, forever — it is MIT.

What you cannot run yourself is a witness with no stake in your answer.

We watch your chain hourly, so the window in which your history could be rewritten is one hour — and we sign that into a receipt your auditor can check without trusting either of us.

15. Appendix — verifying every claim in this document

```
git clone https://github.com/iamGodofall/ABSuite-core && cd ABSuite-core
pnpm install
pnpm verify # build, 1,063 tests, 27 gates
pnpm demo # sign, verify, tamper one byte, watch the chain name it
pnpm gen:layers # the six-of-eight table in §5.1
```

Downloads: <https://api.npmjs.org/downloads/range/2026-08-01:2026-08-28/@absuitecore/capkit>

Every citation in §8 is linked inline. Every **MEASURED** figure came from a command in this repository. Every **REASONED** paragraph shows its method.

If a claim here cannot be checked by running something or following a link, it should not have been written down.